System Teleoperacyjny do Precyzyjnych Prac Elektronicznych

Kompletna Dokumentacja Techniczna

Wersja 1.0 | Data: 2025-10-22 | Autor: Adam Motyl

Spis Treści

1. Przegląd Systemu
2. Specyfikacja Techniczna
3. Komponenty Mechaniczne
4. System Elektroniczny
5. System Wizyjny
6. Interfejs Kontrolny
7. Oprogramowanie
8. Schematy Podłączeń
9. Instrukcja Montażu
10. Konserwacja
11. Rozwiązywanie Problemów
12. Bezpieczeństwo
13. Kod Arduino

1. Przegląd Systemu

1.1 Opis Projektu

System teleoperacyjny przeznaczony do precyzyjnych prac serwisowych w elektronice, szczególnie do lutowania komponentów SMD i manipulacji elementami BGA. Konstrukcja opiera się na sześciuosiowym manipulatorze wykonawczym z dodatkowym skrętnym nadgarstkiem, sterowanym przez dedykowany interfejs kontrolny typu master-slave.

1.2 Główne Cele

- Zwiększenie precyzji pozycjonowania do poziomu 0,5–0,6 mm
- Eliminacja wpływu drgań rąk operatora na jakość pracy
- Poprawa ergonomii stanowiska pracy
- Skrócenie czasu wykonywania skomplikowanych napraw
- Możliwość pracy z różnymi narzędziami (chwytak, lutownica, kamera, mikroskop)

2. Specyfikacja Techniczna

2.1 Parametry Manipulatora

Parametr	Wartość	Opis
Liczba osi	6 (6 DOF)	Pełna swoboda ruchu
Dokładność pozycjonowania	±0,5 mm	W całym zasięgu roboczym
Powtarzalność	±0,2 mm	Standardowa powtarzalność
Rozdzielczość sterowania	0,1 mm	Minimalna rozdzielczość ruchu
Prędkość maksymalna	50 mm/s	Maksymalna prędkość końcówki
Udźwig maksymalny	2 kg	Na końcu ramienia

2.2 Zakresy Ruchu

Oś	Zakres	Typ Serwa	Moment
Oś 1	360°	MG996R	20 kg·cm
Oś 2	−90° do +90°	MG996R	20 kg·cm
Oś 3	−90° do +90°	MG996R	20 kg·cm
Oś 4	−180° do +180°	MG996R	20 kg·cm
Oś 5	−180° do +180°	MG996R	20 kg·cm
Oś 6	−90° do +90°	SG90	1,8 kg·cm

3. Komponenty Mechaniczne

3.1 Struktura Główna

Konstrukcja opiera się na trzech segmentach ramienia zakończonych nadgarstkiem z wymiennym chwytakiem. Wszystkie elementy są zaprojektowane z myślą o łatwości montażu i konserwacji.

- **Podstawa:** element mocujący do stołu roboczego z regulacją wysokości
- **Ramiona:** 3 segmenty z przegubami kulistymi
- **Nadgarstek:** przegub trójosiowy z możliwością skrętu
- **Chwytak:** system szybkomocujących adapterów narzędzi

3.2 Materiały

Element	Materiał	Metoda Produkcji	Uwagi
Główne elementy	PLA	Druk 3D	Łatwość druku, dobra sztywność
Elementy obciążone	PETG	Druk 3D	Wysoka wytrzymałość, elastyczność
Łożyska	Stal	Gotowe 608ZZ	Płynność ruchu, trwałość
Osie	Stal nierdzewna	M8, M6, M4	Wytrzymałość, precyzja

3.3 System Napędowy

- **5× MG996R (20 kg·cm)** – osie główne (podstawa, ramię, przedramię, nadgarstek, skręt)
- **1× SG90 (1,8 kg·cm)** – chwytak

4. System Elektroniczny

4.1 Mikrokontrolery

- **Master:** Arduino Nano lub ESP32 DevKit V1 (manipulator)
- **Slave:** Arduino Mega 2560 lub ESP32 DevKit V1 (interfejs kontrolny)

4.2 Sterowniki i Czujniki

- **PCA9685** – 16-kanałowy sterownik PWM (I2C) do zarządzania serwomechanizmami
- **6× Enkodery KY-040** – precyzyjny odczyt pozycji dla interfejsu kontrolnego
- **2× TTP223** – czujniki dotyku pojemnościowego dla dodatkowych funkcji
- **4× Micro Switch** – przyciski Start, Stop, Emergency, Tool Change

4.3 Zasilanie

Zasilacz	Napięcie/Prąd	Przeznaczenie	Uwagi
Główny	5V / 10A	Serwomechanizmy, arduino, enkodery, czujniki	Stabilne zasilanie dla serw
LED Ring	12V / 2A	Oświetlenie pierścieniowe	Wysokiej jakości oświetlenie

4.4 Komunikacja

- **Protokół:** UART Master–Slave z protokołem JSON
- **Prędkość:** 115200 bps
- **Format:** JSON (pozycje, stany, błędy, komendy)

5. System Wizyjny

5.1 Kamera

- **Typ:** USB HD 1080p z autofocusem
- **Alternatywa:** Raspberry Pi Camera V2
- **Rozdzielczość:** 1920×1080 @ 30 fps
- **Zakres ogniskowej:** automatyczny z możliwością ręcznej regulacji

5.2 Optyka i Oświetlenie

- **Powiększenie:** 2–10× (soczewka powiększająca)
- **Oświetlenie:** pierścień LED z regulacją jasności (0-100%)
- **Uwagi:** równomierne światło znacznie poprawia komfort i precyzję pracy

5.3 Oprogramowanie Wizyjne

- Podgląd wideo w czasie rzeczywistym z minimalnym opóźnieniem
- Nakładki pomocnicze (siatka, celownik, linie pomocnicze)
- Rejestracja sesji roboczych do analizy i szkoleń

6. Interfejs Kontrolny

6.1 Konstrukcja i Elementy Sterujące

Panel sterujący typu **master–slave** zapewnia bezpośrednie odwzorowanie ruchów operatora z możliwością skalowania precyzji.

- 6 osi z enkoderami KY-040 (precyzyjny odczyt pozycji)
- Przyciski: Start, Stop, Emergency, Tool Change
- Czujniki dotyku TTP223 dla dodatkowych funkcji

7. Oprogramowanie

7.1 Algorytmy Kinematyki

- **Kinematyka prosta:** przeliczenie kątów osi na współrzędne narzędzia
- **Kinematyka odwrotna:** obliczanie pozycji stawów dla zadanej pozycji końcówki
- **Skalowanie ruchu:** regulowane 1:1 do 1:10 (dla różnych poziomów precyzji)

7.2 System Sterowania

- **Regulatory PID** dla każdej osi (niezależna regulacja)
- **Automatyczna kalibracja** po uruchomieniu systemu
- **Filtrowanie sygnałów** dla płynności ruchu

7.3 Interfejs Użytkownika

Aplikacja desktopowa (Python/C++) zapewnia intuicyjną obsługę i wizualizację.

- Kalibracja systemu (automatyczna i ręczna)
- Ustawienia skalowania i trybów pracy
- Wybór narzędzia i monitoring stanu
- Rejestracja i analiza sesji roboczych

8. Schematy Podłączeń

8.1 Schemat Główny Systemu

```
┌───────────────────────────────────────────────────────────────────┐
│                    SYSTEM TELEOPERACYJNY                          │
├───────────────────────────────────────────────────────────────────┤
│                                                                   │
│  ┌─────────────┐   ┌─────────────┐   ┌─────────────┐              │
│  │ INTERFEJS   │   │   MASTER    │   │   SLAVE     │              │
│  │ KONTROLNY   │◄─►│ ARDUINO     │◄─►│ ARDUINO     │              │
│  │             │   │ NANO/ESP32  │   │ MEGA/ESP32  │              │
│  └─────────────┘   └─────────────┘   └─────────────┘              │
│         │                                   │                     │
│         │                                   │                     │
│         │                                   ▼                     │
│  ┌─────────────┐                     ┌─────────────┐              │
│  │ 6x ENKODERY │                     │ PCA9685     │              │
│  │ KY-040      │                     │ PWM DRIVER  │              │
│  └─────────────┘                     └─────────────┘              │
│         │                                   │                     │
│         │                                   │                     │
│         │                                   ▼                     │
│  ┌─────────────┐                     ┌─────────────┐              │
│  │ 2x TTP223   │                     │ 6x SERVO    │              │
│  │ CZUJNIKI    │                     │ MG996R/SG90 │              │
│  └─────────────┘                     └─────────────┘              │
│                                                                   │
└───────────────────────────────────────────────────────────────────┘
```

8.2 Podłączenia Enkoderów

```
ENKODER KY-040 (każdy):
┌─────────────┐
│     VCC     │ ────► 5V
│     GND     │ ────► GND
│     CLK     │ ────► Pin D (np. D2)
│     DT      │ ────► Pin D+1 (np. D3)
│     SW      │ ────► (opcjonalnie)
└─────────────┘

OŚ 1: CLK→D2, DT→D3
OŚ 2: CLK→D4, DT→D5
OŚ 3: CLK→D6, DT→D7
OŚ 4: CLK→D8, DT→D9
OŚ 5: CLK→D10, DT→D11
OŚ 6: CLK→D12, DT→D13
```

8.3 Podłączenia Serwomechanizmów

```
SERWOMECHANIZMY:
┌──────────────────────────────────────────────────────────────┐
│                                                              │
│         OŚ 1                 OŚ 2                            │
│   ┌─────────────┐      ┌─────────────┐                       │
│   │ MG996R      │      │ MG996R      │                       │
│   │ VCC → 5V    │      │ VCC → 5V    │                       │
│   │ GND → GND   │      │ GND → GND   │                       │
│   │ SIG → PWM0  │      │ SIG → PWM1  │                       │
│   └─────────────┘      └─────────────┘                       │
│                                                              │
│         OŚ 3                 OŚ 4                            │
│   ┌─────────────┐      ┌─────────────┐                       │
│   │ MG996R      │      │ MG996R      │                       │
│   │ VCC → 5V    │      │ VCC → 5V    │                       │
│   │ GND → GND   │      │ GND → GND   │                       │
│   │ SIG → PWM2  │      │ SIG → PWM3  │                       │
│   └─────────────┘      └─────────────┘                       │
│                                                              │
│         OŚ 5                 OŚ 6                            │
│   ┌─────────────┐      ┌─────────────┐                       │
│   │ MG996R      │      │ SG90        │                       │
│   │ VCC → 5V    │      │ VCC → 5V    │                       │
│   │ GND → GND   │      │ GND → GND   │                       │
│   │ SIG → PWM4  │      │ SIG → PWM5  │                       │
│   └─────────────┘      └─────────────┘                       │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

9. Instrukcja Montażu

9.1 Przygotowanie Komponentów

Części Mechaniczne:

- **Elementy drukowane 3D** - sprawdź jakość wydruku
- **5x Serwomechanizmy MG996R** (20kg/cm) - osie główne
- **1x Serwomechanizm SG90** (1.8kg/cm) - chwytak
- **Śruby M3, M4, M6, M8** (zestaw) - różne długości
- **Nakrętki i podkładki** (zestaw) - stalowe i nylonowe
- **Tulejki dystansowe** (różne długości) - do precyzyjnego pozycjonowania

Części Elektroniczne:

- **1x Arduino Mega 2560 / Arduino Nano** (lub 2x ESP32) - Master i Slave
- **1x PCA9685** (sterownik PWM) - 16-kanałowy I2C
- **6x Enkodery KY-040** - precyzyjny odczyt pozycji
- **2x Czujniki TTP223** - dotyk pojemnościowy
- **4x Przyciski micro switch** - Start, Stop, Emergency, Tool
- **Zasilacze 5V 10A, 12V 2A, /strong> - różne obwody**
- **Przewody i złącza - JST, XT60, przewody połączeniowe**

9.2 Montaż Mechaniczny

1. Przygotowanie podstawy
    - Sprawdź czy elementy 3D są wydrukowane poprawnie
    - Usuń podpory i wygładź powierzchnie
    - Sprawdź otwory na śruby
2. Montaż serwomechanizmu osi 1
    - Zamontuj MG996R w podstawie
    - Użyj śrub M3 do mocowania
    - Sprawdź czy serwo obraca się swobodnie
3. Montaż ramienia
    - Segment 1 (ramię) - zamontuj MG996R
    - Segment 2 (przedramię) - zamontuj MG996R
    - Segment 3 (nadgarstek) - zamontuj MG996R

9.3 Montaż Elektroniczny

1. Przygotowanie płytek
    - Arduino Master (Interfejs) - zamontuj na płytce prototypowej
    - Arduino Slave (Manipulator) - zamontuj PCA9685
2. Okablowanie interfejsu kontrolnego
    - Podłącz 6 enkoderów KY-040
    - Podłącz 2 czujniki TTP223
    - Podłącz 4 przyciski
3. Okablowanie manipulatora
    - Podłącz PCA9685 przez I2C
    - Podłącz 7 serwomechanizmów

10. Konserwacja

10.1 Regularne Przeglądy

Częstotliwość	Zadania	Opis
Codziennie	Sprawdzenie zer mechanicznych	Sprawdź czy wszystkie osie są w pozycji zerowej
Tygodniowo	Pełna kalibracja systemu	Wykonaj pełną kalibrację wszystkich osi
Miesięcznie	Smarowanie łożysk	Nasmaruj wszystkie łożyska i przeguby

10.2 Procedury Kalibracji

1. Zera mechaniczne - ustaw wszystkie osie w pozycji zerowej
2. Zakresy ruchu - sprawdź limity każdej osi
3. Skalowanie - kalibracja współczynników skalowania
4. Testy - weryfikacja precyzji i powtarzalności

11. Rozwiązywanie Problemów

11.1 Typowe Problemy

Problem	Przyczyna	Rozwiązanie
Drgania	Luz łożysk, nieprawidłowe mocowanie	Sprawdź luz łożysk, wyreguluj serwa
Niedokładność	Luz w połączeniach, nieprawidłowa kalibracja	Sprawdź luz w połączeniach, wyreguluj serwa
Brak komunikacji	Błędne okablowanie UART	Sprawdź okablowanie UART, sprawdź prędkość
Przegrzewanie serw	Przeciążenie, złe zasilanie	Sprawdź obciążenie, sprawdź zasilanie

11.2 Diagnostyka

- Testy osi: Sprawdź każdą oś osobno (funkcja diagnostyczna w oprogramowaniu)
- Monitoring: Sprawdź temperaturę i prądy (wbudowane czujniki)
- Logi: Analiza logów błędów (plik debug.txt)

12. Bezpieczeństwo

12.1 Zabezpieczenia

- Emergency Stop: Przycisk awaryjnego zatrzymania
- Limity: Mechaniczne i programowe limity ruchu
- Monitoring: Ciągłe monitorowanie stanu systemu (watchdog)

12.2 Procedury Bezpieczeństwa

1. Przed pracą: Sprawdź wszystkie systemy
2. Podczas pracy: Ciągły monitoring operatora
3. Po pracy: Bezpieczne wyłączenie i zabezpieczenie

WAŻNE: Zawsze używaj przycisku Emergency Stop w przypadku awarii lub niebezpiecznej sytuacji.

13. Kod Arduino

13.1 Arduino Master (Interfejs Kontrolny)

Uwaga: Kod zawiera pełną implementację komunikacji Master-Slave z protokołem JSON. Piny SoftwareSerial (14, 15) są kompatybilne z Arduino Mega.

```
/*
 * System Teleoperacyjny - Arduino Master (Interfejs Kontrolny)
 * Wersja: 1.0
 * Data: 2025-01-13
 *
 * Funkcje:
 * - Odczyt enkoderów (6 osi)
 * - Obsługa czujników dotyku
 * - Obsługa przycisków
 * - Komunikacja UART z Slave
 * - Przesyłanie danych pozycji
 */

#include
#include
#include

// Definicje pinów enkoderów
Encoder enc1(2, 3);   // Oś 1 (podstawa)
Encoder enc2(4, 5);   // Oś 2 (ramię)
Encoder enc3(6, 7);   // Oś 3 (przedramię)
Encoder enc4(8, 9);   // Oś 4 (nadgarstek)
Encoder enc5(10, 11); // Oś 5 (skręt)
Encoder enc6(12, 13); // Oś 6 (chwytak)

// Definicje pinów czujników
#define TOUCH_SENSOR_1 A0
#define TOUCH_SENSOR_2 A1

// Definicje pinów przycisków
#define BUTTON_START A2
#define BUTTON_STOP A3
#define BUTTON_EMERGENCY A4
#define BUTTON_TOOL A5

// Komunikacja UART z Slave
SoftwareSerial slaveSerial(14, 15); // RX, TX (piny 14, 15 dla SoftwareSerial)

// Zmienne globalne
long enc1Pos = 0, enc2Pos = 0, enc3Pos = 0;
long enc4Pos = 0, enc5Pos = 0, enc6Pos = 0;
long enc1New = 0, enc2New = 0, enc3New = 0;
long enc4New = 0, enc5New = 0, enc6New = 0;

// Współczynniki skalowania (regulowalne)
float scaleFactor = 1.0; // 1:1 do 1:10
bool emergencyStop = false;
bool systemActive = false;

// Czasy
unsigned long lastSendTime = 0;
const unsigned long sendInterval = 50; // 20Hz

void setup() {
  // Inicjalizacja portu szeregowego
  Serial.begin(115200);
  slaveSerial.begin(115200);

  // Konfiguracja pinów
  pinMode(TOUCH_SENSOR_1, INPUT);
  pinMode(TOUCH_SENSOR_2, INPUT);
  pinMode(BUTTON_START, INPUT_PULLUP);
  pinMode(BUTTON_STOP, INPUT_PULLUP);
  pinMode(BUTTON_EMERGENCY, INPUT_PULLUP);
  pinMode(BUTTON_TOOL, INPUT_PULLUP);

  // Inicjalizacja enkoderów
  enc1.write(0);
  enc2.write(0);
  enc3.write(0);
  enc4.write(0);
  enc5.write(0);
  enc6.write(0);

  Serial.println("System Teleoperacyjny - Master uruchomiony");
  Serial.println("Czekam na połączenie z Slave...");

  // Czekaj na połączenie z Slave
  while (!slaveSerial.available()) {
    delay(100);
  }

  Serial.println("Połączenie z Slave nawiązane");
  sendSystemStatus();
}

void loop() {
  // Sprawdź przyciski
  checkButtons();

  // Sprawdź czujniki dotyku
  checkTouchSensors();

  // Odczytaj enkodery
  readEncoders();

  // Wyślij dane do Slave
  if (millis() - lastSendTime >= sendInterval) {
    sendPositionData();
    lastSendTime = millis();
  }

  // Sprawdź odpowiedź od Slave
  checkSlaveResponse();

  // Krótkie opóźnienie
  delay(10);
}

// [Pozostałe funkcje...]
```

13.2 Arduino Slave (Manipulator)

Uwaga: Kod zawiera pełną implementację sterowania serwomechanizmami przez PCA9685 z płynnym przejściem między pozycjami.

```
/*
 * System Teleoperacyjny - Arduino Slave (Manipulator)
 * Wersja: 1.0
 * Data: 2025-01-13
 *
 * Funkcje:
 * - Sterowanie serwomechanizmami (6 osi)
 * - Komunikacja UART z Master
 * - Obsługa PCA9685 PWM Driver
 * - Monitoring stanu systemu
 */

#include
#include
#include

// Komunikacja UART z Master
SoftwareSerial masterSerial(14, 15); // RX, TX (piny 14, 15 dla SoftwareSerial)

// PCA9685 PWM Driver
Adafruit_PWMServoDriver pwm = Adafruit_PWMServoDriver();

// Definicje serwomechanizmów
#define SERVO_1_CHANNEL 0   // Oś 1 (podstawa) - MG996R
#define SERVO_2_CHANNEL 1   // Oś 2 (ramię) - MG996R
#define SERVO_3_CHANNEL 2   // Oś 3 (przedramię) - MG996R
#define SERVO_4_CHANNEL 3   // Oś 4 (nadgarstek) - MG996R
#define SERVO_5_CHANNEL 4   // Oś 5 (skręt) - MG996R
 #define SERVO_6_CHANNEL 5   // Oś 6 (chwytak) - SG90

// Parametry serwomechanizmów
#define SERVO_FREQ 50        // Częstotliwość PWM (50Hz)
#define SERVO_MIN_PULSE 150  // Minimalny impuls (0.75ms)
#define SERVO_MAX_PULSE 600  // Maksymalny impuls (3.0ms)

 // Zmienne globalne
 int currentServoPos[6] = {90, 90, 90, 90, 90, 90}; // Pozycje serw (0-180°)
 int targetServoPos[6] = {90, 90, 90, 90, 90, 90};  // Docelowe pozycje
bool systemActive = false;
bool emergencyStop = false;
bool calibrationMode = false;

// Czasy
unsigned long lastUpdateTime = 0;
const unsigned long updateInterval = 20; // 50Hz
unsigned long lastResponseTime = 0;
const unsigned long responseInterval = 1000; // 1Hz

void setup() {
  // Inicjalizacja portu szeregowego
  Serial.begin(115200);
  masterSerial.begin(115200);

  // Inicjalizacja PCA9685
  pwm.begin();
  pwm.setOscillatorFrequency(27000000);
  pwm.setPWMFreq(SERVO_FREQ);

   // Ustaw wszystkie serwa w pozycji neutralnej
   for (int i = 0; i < 6; i++) {
     setServoPosition(i, 90);
   }

  Serial.println("System Teleoperacyjny - Slave uruchomiony");
  Serial.println("Czekam na połączenie z Master...");

  // Czekaj na połączenie z Master
  while (!masterSerial.available()) {
    delay(100);
  }

  Serial.println("Połączenie z Master nawiązane");
  sendStatusResponse();
}

void loop() {
  // Sprawdź komunikację z Master
  checkMasterCommunication();

  // Aktualizuj pozycje serw
  if (millis() - lastUpdateTime >= updateInterval) {
    updateServoPositions();
    lastUpdateTime = millis();
  }

  // Wyślij odpowiedź statusu
  if (millis() - lastResponseTime >= responseInterval) {
    sendStatusResponse();
    lastResponseTime = millis();
  }

  // Krótkie opóźnienie
  delay(5);
}

// [Pozostałe funkcje...]
```

14. Lista Komponentów

14.1 Mikrokontrolery

- 2x Arduino Mega 2560 (lub ESP32 DevKit V1)
- 2x Płytki prototypowe 400 punktów
- 2x Obudowy do mikrokontrolerów

14.2 Sterowniki i Moduły

- 1x PCA9685 - 16-kanałowy sterownik PWM I2C
- 6x KY-040 - Enkodery obrotowe
- 2x TTP223 - Czujniki dotyku pojemnościowe
- 4x Micro Switch - Przyciski

14.3 Serwomechanizmy

- 5x MG996R - Serwomechanizmy 20kg/cm (osie główne: podstawa, ramię, przedramię, nadgarstek, skręt)
- 1x SG90 - Serwomechanizm 1.8kg/cm (chwytak)

14.4 Zasilanie

- 2x Zasilacz 5V 5A - Elektronika
- 1x Zasilacz 12V 2A - LED Ring

14.5 Okablowanie

- Przewody połączeniowe 1x0.5mm² (długość 50m)
- Przewody zasilające 2x0.75mm² (długość 20m)
- Złącza JST 2.54mm (20 sztuk)
- Złącza XT60 (4 sztuki)
- Taśma izolacyjna (1 rolka)

15. Harmonogram Realizacji

15.1 Status Projektu

Etap	Status	Opis
I - Analiza i Projektowanie	✅ Ukończony	Badanie potrzeb, wymagania techniczne
II - Projekt Mechaniczny	✅ Ukończony	Projektowanie CAD, analiza wytrzymałościowa
III - Realizacja Mechaniczna	✅ Ukończony	Produkcja elementów, montaż napędu
IV - System Sterowania	🔄 W trakcie	Integracja elektroniczna, komunikacja
V - System Wizyjny	🔄 W trakcie	Integracja kamery, oświetlenie
VI - Testy i Optymalizacja	⏳ Planowane	Weryfikacja funkcjonalna, optymalizacja
VII - Dokumentacja	✅ Ukończony	Dokumentacja techniczna i użytkownika

Następne kroki:

1. Dokończenie systemu sterowania (Etap IV)
2. Integracja systemu wizyjnego (Etap V)
3. Testy i optymalizacja (Etap VI)

16. Kontakt i Wsparcie

16.1 Dokumentacja

- Dokumentacja Techniczna: Kompletna specyfikacja systemu
- Schematy Podłączeń: Szczegółowe schematy elektryczne
- Instrukcja Montażu: Krok po kroku instrukcja montażu
- Kod Arduino: Kompletne oprogramowanie

16.2 Rozwiązywanie Problemów

- Sprawdź sekcję "Rozwiązywanie Problemów" w tej dokumentacji
- Sprawdź logi systemu w przypadku błędów
- Wykonaj testy diagnostyczne

Wersja dokumentacji: 1.0
Data: 2025-01-13
Status: Kompletna dokumentacja techniczna
Autor: Adam Motyl

Dokumentacja została przygotowana na podstawie specyfikacji technicznej systemu teleoperacyjnego do precyzyjnych prac elektronicznych.